

云计算导论Lab6

Cloud-Native Automated Deployment

Laboratory Report of Nanjing University of Information Science and Technology

Experiment Title: Cloud-Native Automated Deployment	Date: 2026.5.24	Class: 2023	Name: Linjia Guo
ID: 202383910005	Course: Cloud Computing Lab	Report Type: Laboratory Report	Status: Completed

1. Experimental Purpose

The purpose of this experiment is to master the basic process of cloud-native automated deployment through GitHub, GitHub Actions and GitHub Pages. Through practical operation, I learned how the cloud platform completes resource acquisition, automated pipeline triggering, project construction and static website release without local server configuration.

This experiment combines practical operation with core cloud computing concepts, helping me deeply understand **Platform as a Service (PaaS)**, managed pipeline service, on-demand computing and continuous deployment.

2. Experiment Contents

This experiment simulates a complete cloud deployment life cycle, and the main tasks are as follows:

- Fork the specified GitHub repository to my personal GitHub account.
- Enable GitHub Pages and set GitHub Actions as the deployment source.
- Edit the index.html file directly in the browser and submit the modification.

- Observe the running process of Cloud-Deployment-Pipeline in GitHub Actions.
- Verify the online status of the website after the deployment is successful.
- Personalize the page content by modifying HTML/CSS codes and adding multimedia elements.

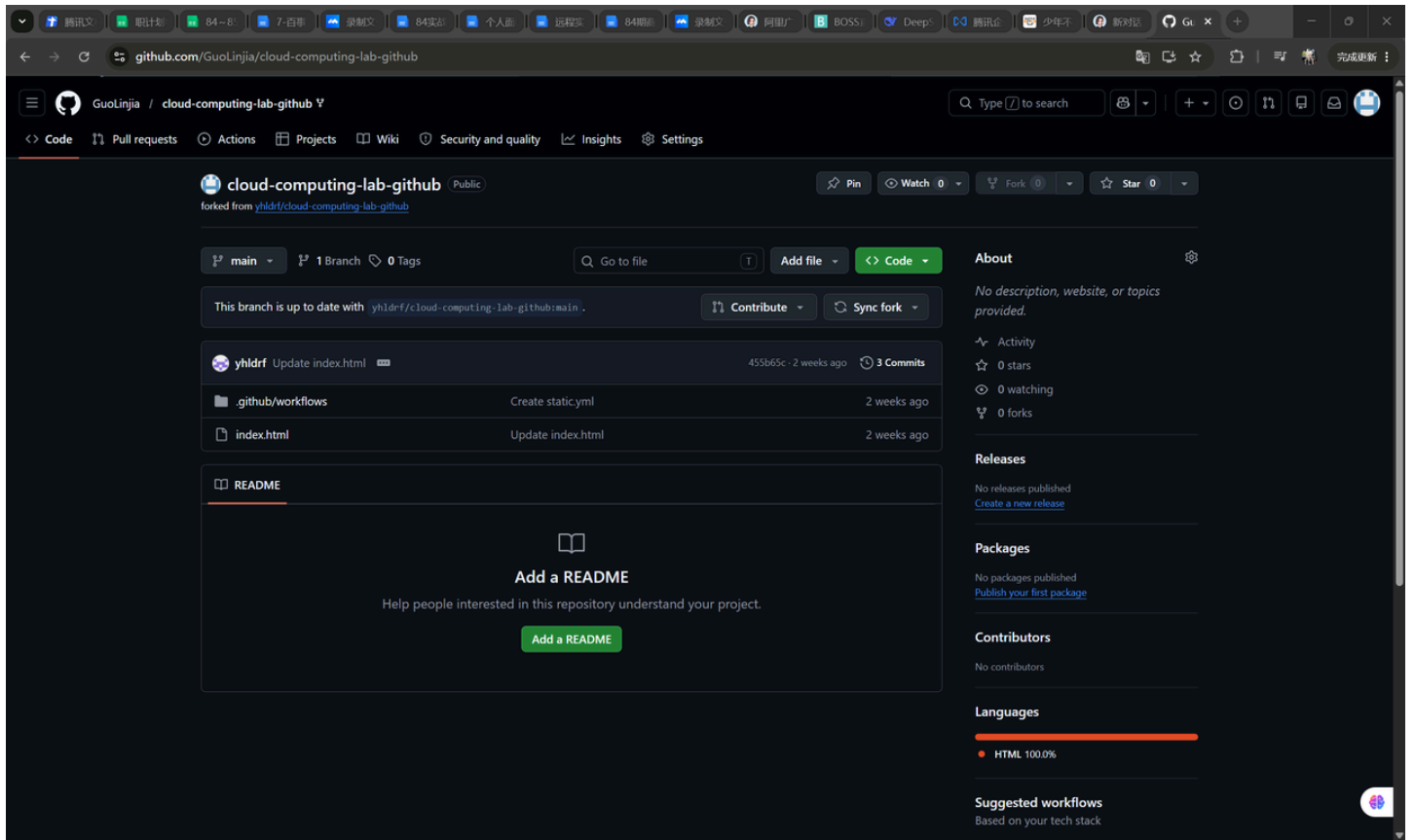
3. Detailed Steps of the Experiment

I completed all operations through the GitHub web interface, and recorded the experimental process and corresponding screenshots in strict accordance with the steps.

Step 1: Resource Acquisition

I logged into my personal GitHub account, opened the experimental repository <https://github.com/yhldrf/cloud-computing-lab-github>, and clicked the **Fork** button to copy the project to my account. After the Fork was completed, I confirmed that all files (including index.html and workflow configuration files) were successfully copied to my personal repository.

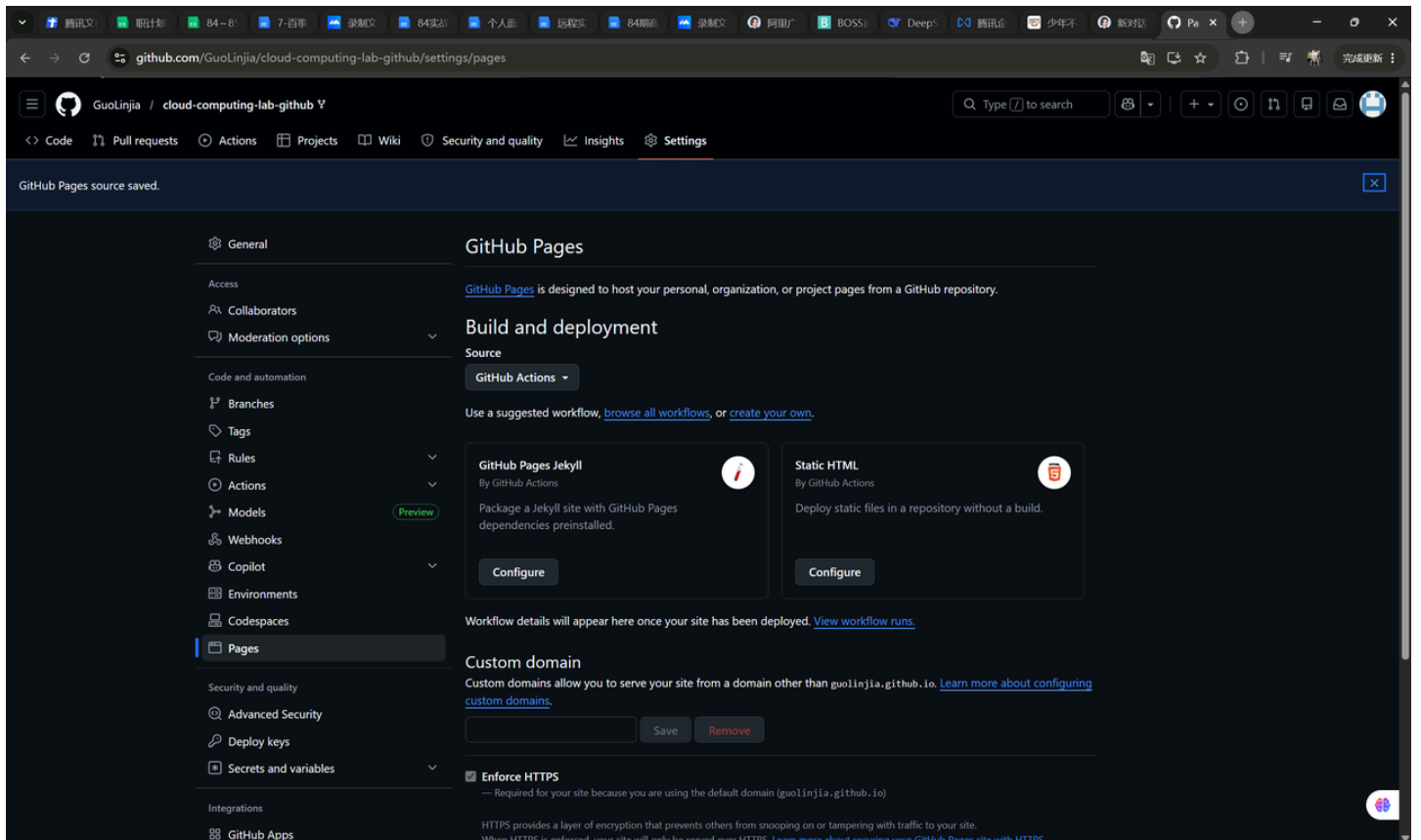
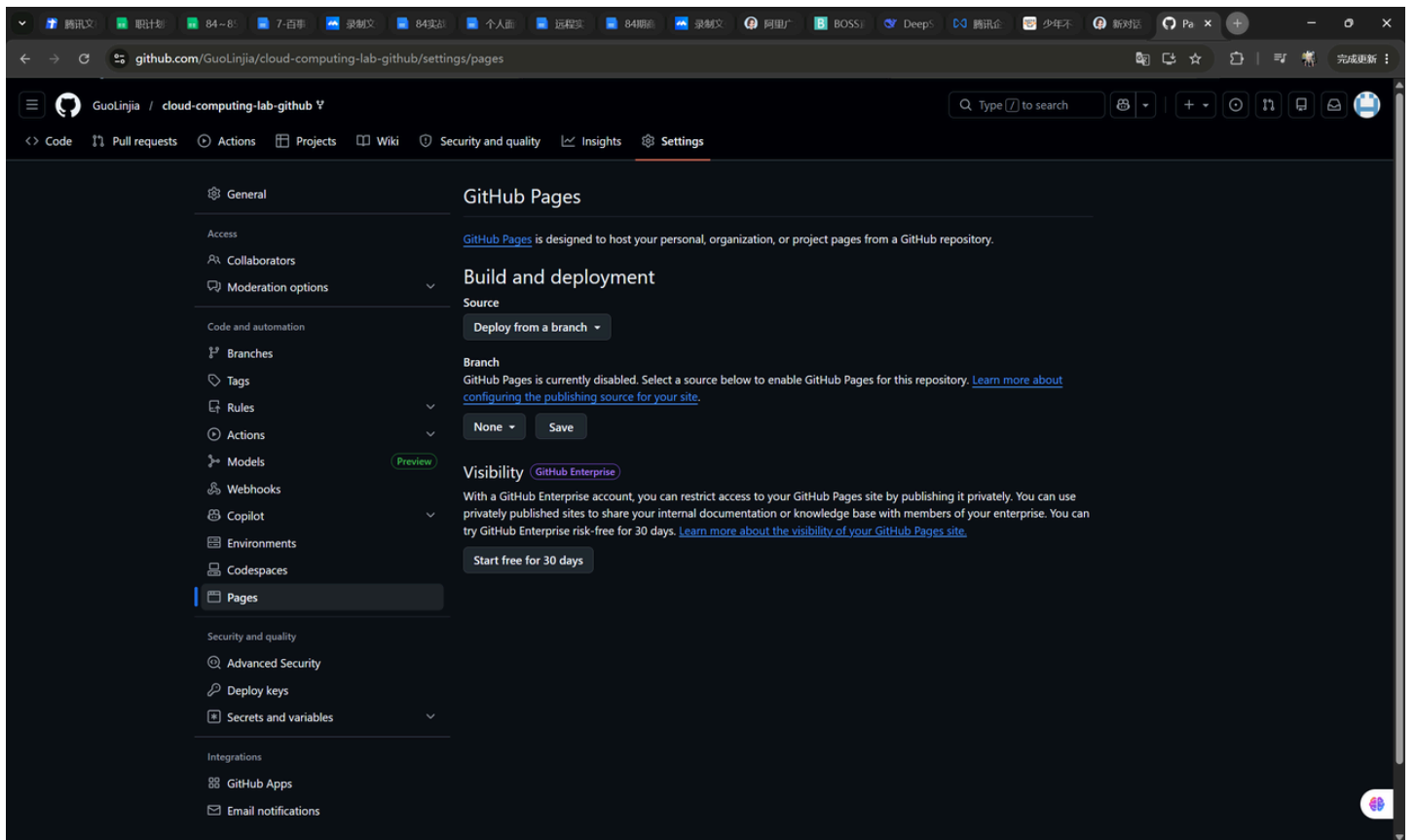
My operation	Cloud concept reflected
Forked the experimental repository to obtain an independent editable project copy.	This step is the resource acquisition in the cloud ecosystem. I obtained an independent project instance, which can be edited and deployed without affecting the original repository.



Step 2: Enabling the Hosting Service

I entered the repository page, opened **Settings > Pages**, and set the deployment source to **GitHub Actions** in the Build and deployment area. This setting switches the deployment mode from manual upload to automated pipeline deployment.

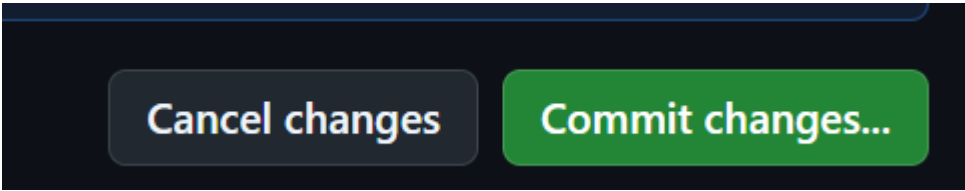
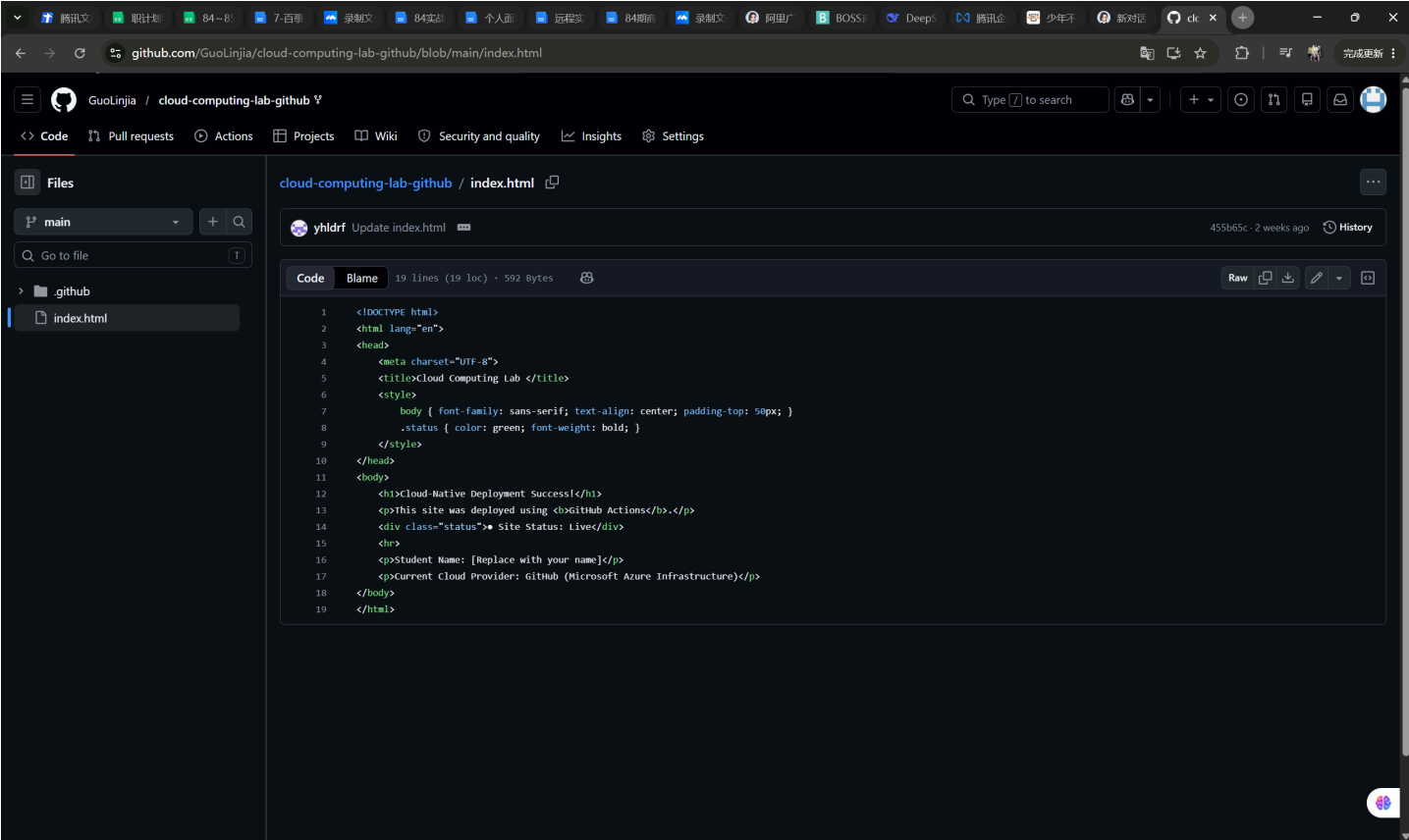
My operation	Cloud concept reflected
Configured GitHub Pages and selected GitHub Actions as the deployment source.	The deployment mode is switched to a managed pipeline service. GitHub provides a complete hosting and deployment environment, and I only need to maintain project content.

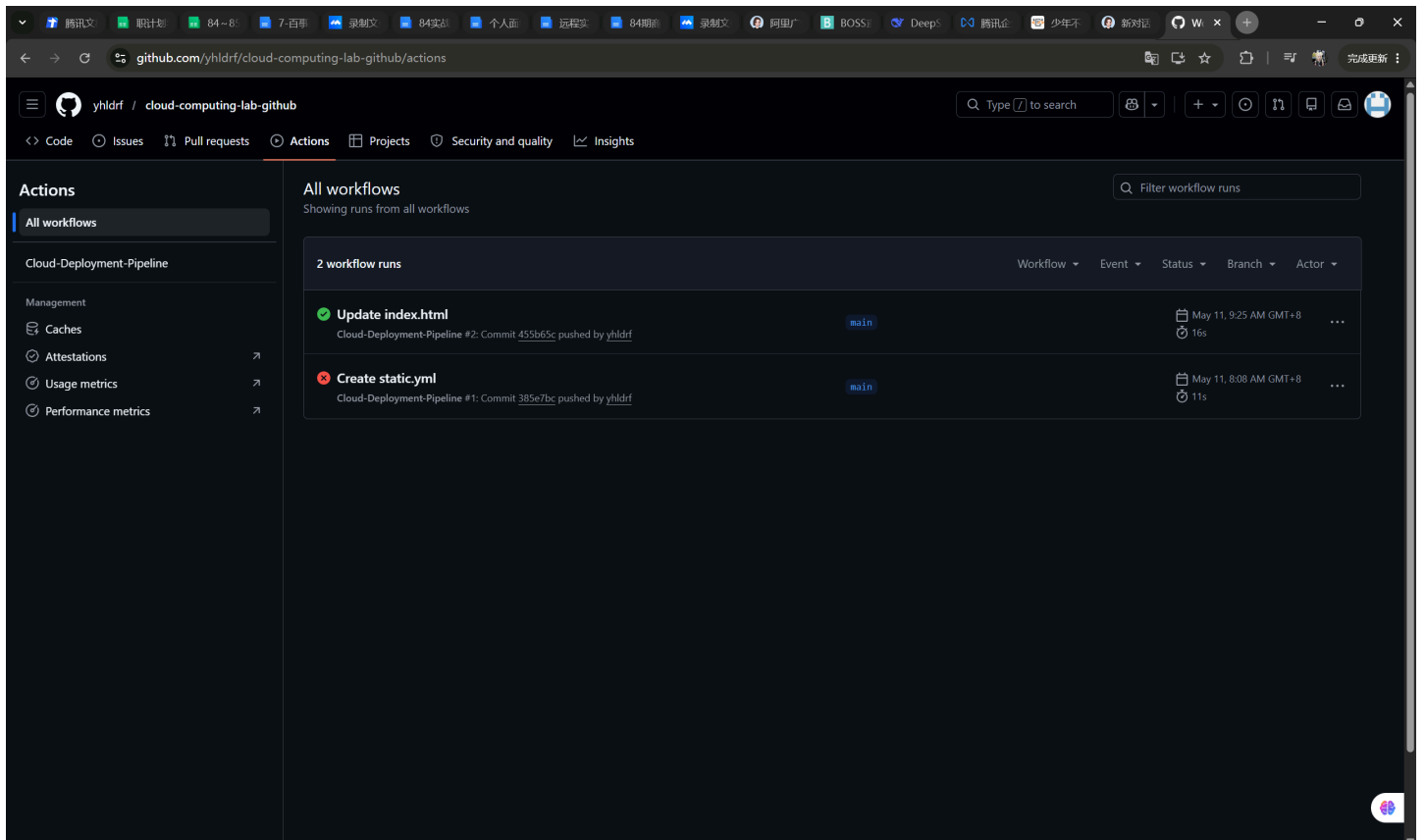
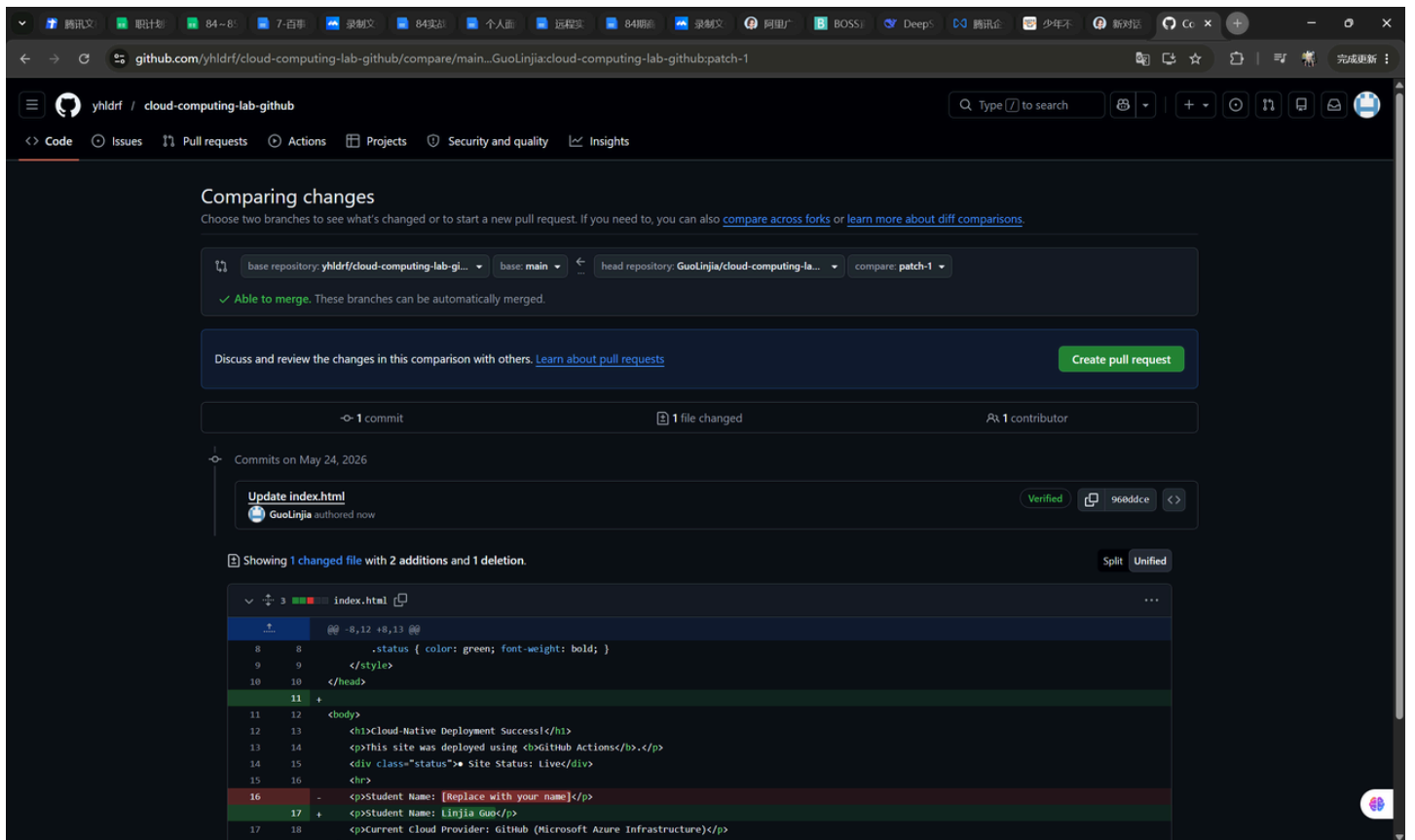


Step 3: Triggering Cloud Automation

I opened index.html in the GitHub online editor, replaced [Enter Your Name Here] with my real name, and clicked **Commit changes** to submit the modification. The submission operation immediately triggered the GitHub Actions workflow, and the cloud platform automatically started the deployment pipeline.

My operation	Cloud concept reflected
Edited index.html online and submitted the modification.	GitHub Actions starts a virtualized execution environment on demand, which reflects the on-demand computing capability of the cloud platform.

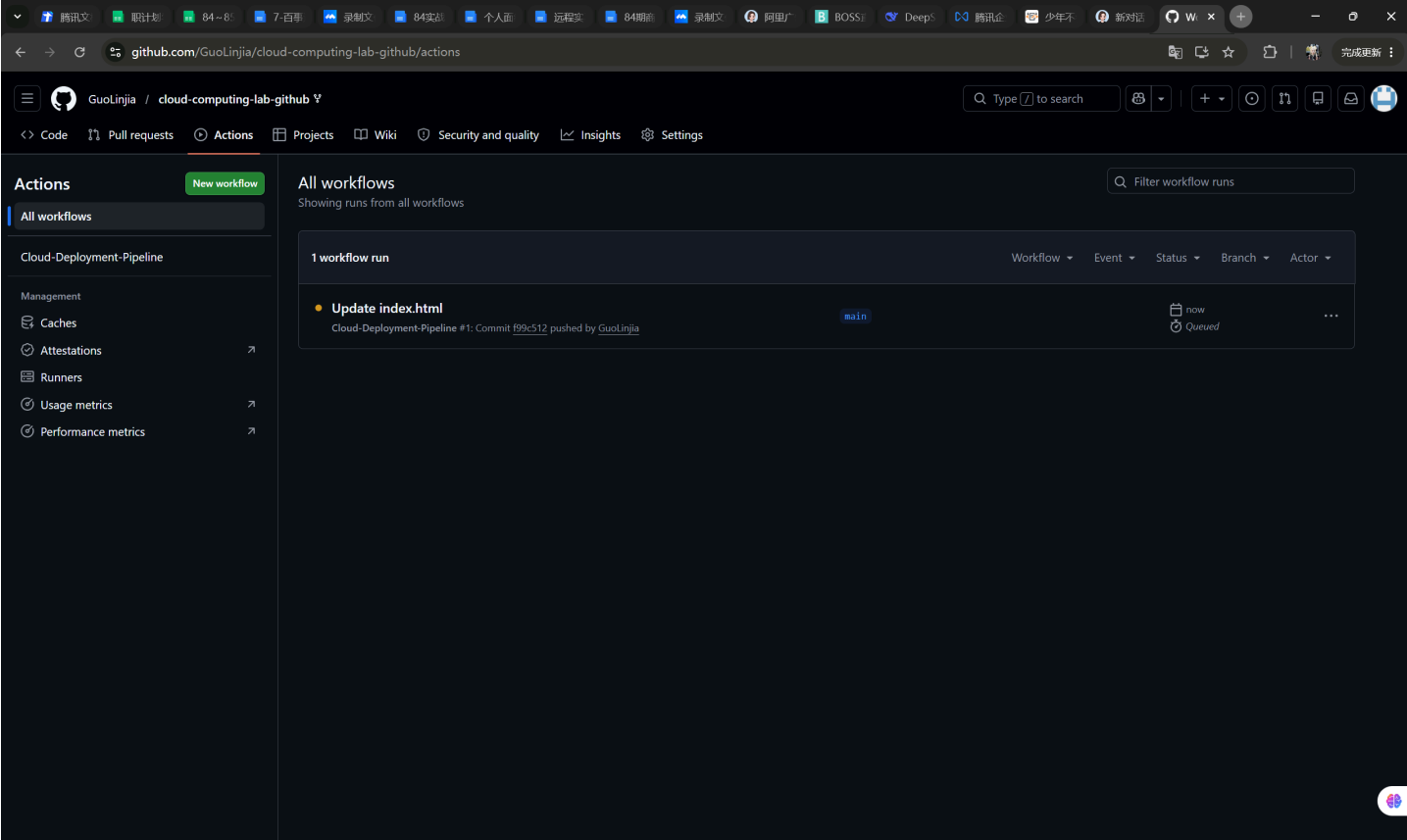


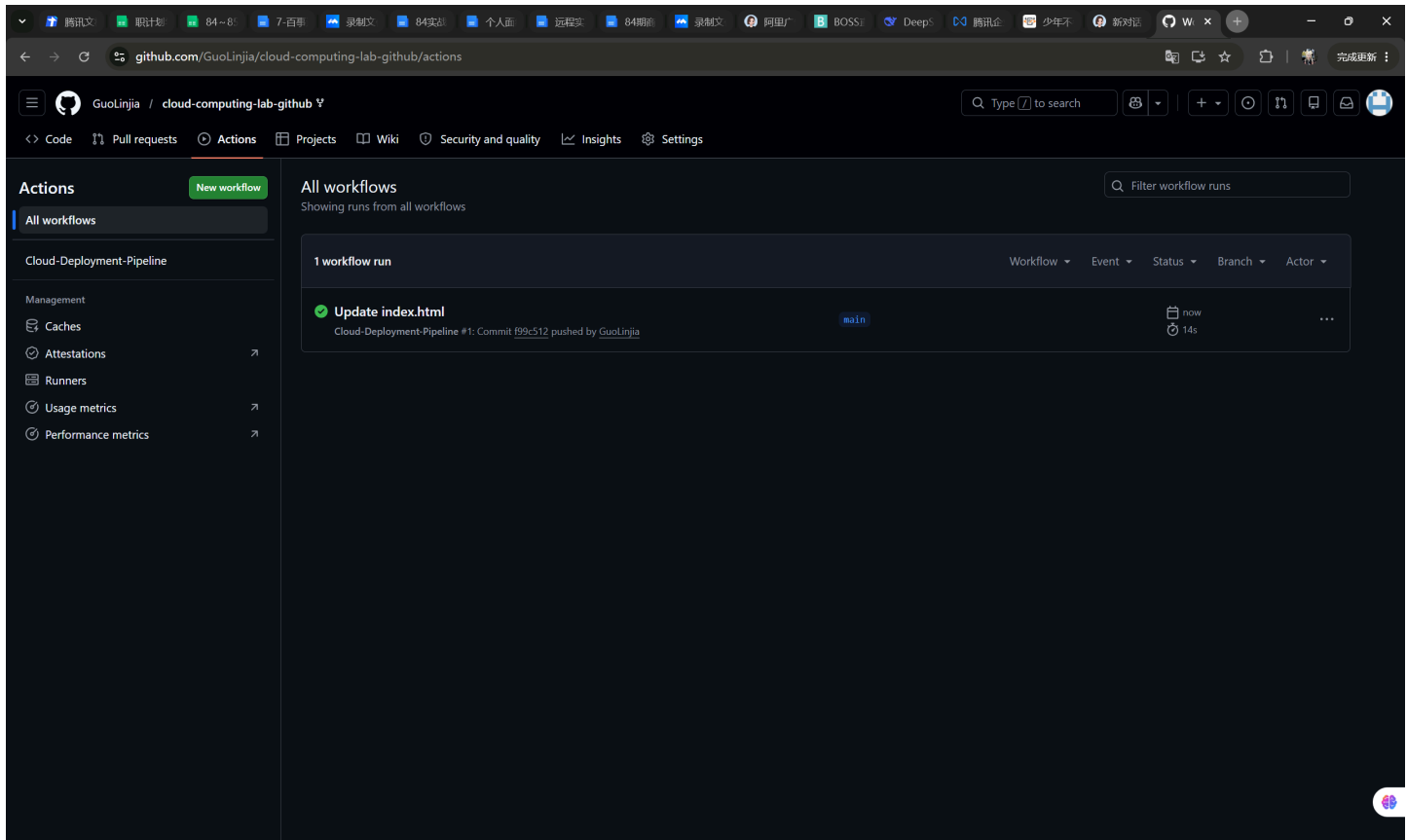


Step 4: Observing the Deployment Pipeline

After submitting the code, I switched to the **Actions** tab and observed the running status of the **Cloud-Deployment-Pipeline** workflow. When the workflow showed a green check mark, it meant the deployment task was completed successfully.

My operation	Cloud concept reflected
Monitored the workflow running status in the Actions tab.	This step reflects continuous integration and continuous deployment. Each submission will automatically trigger a new cloud build and deployment.

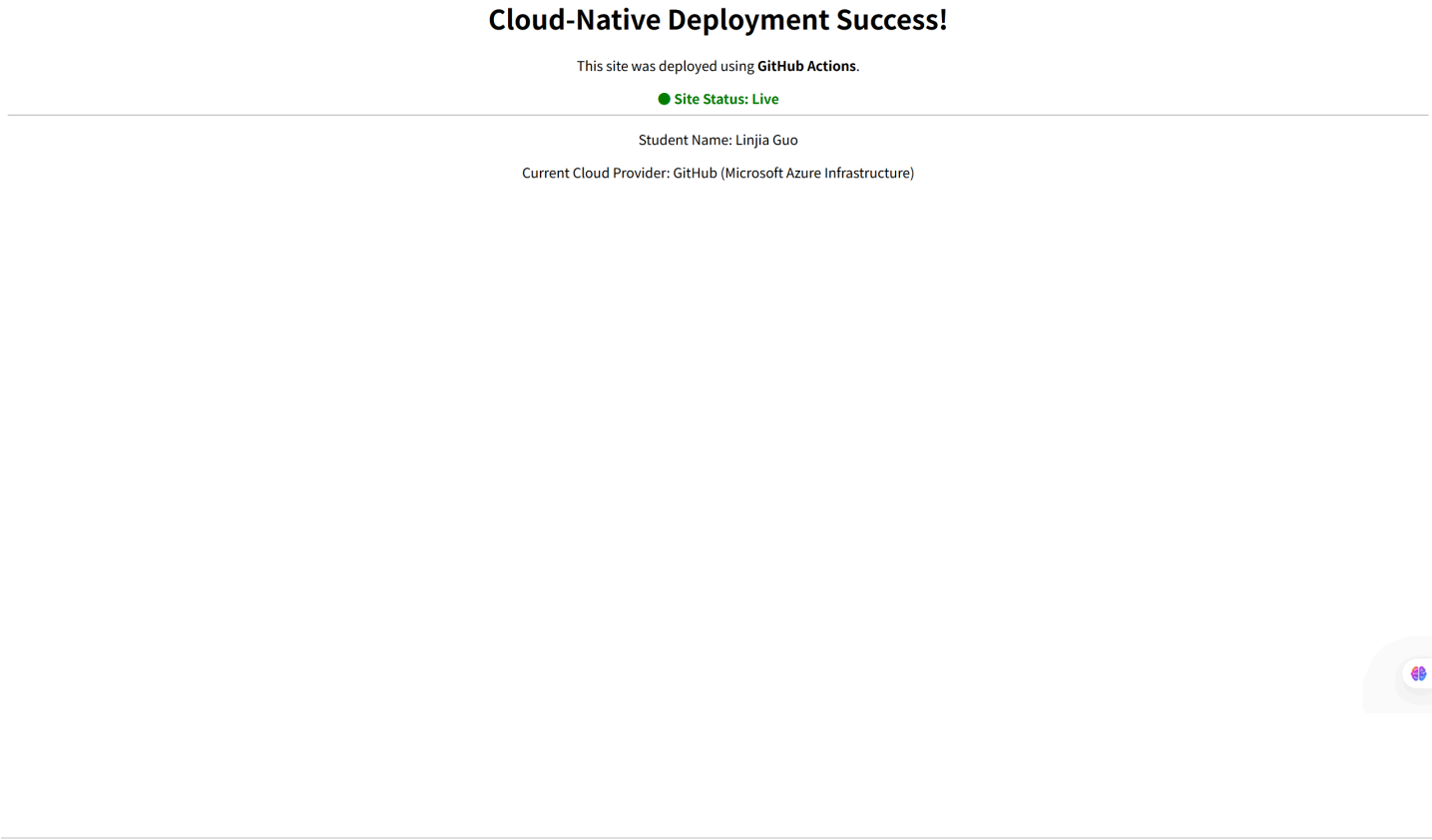
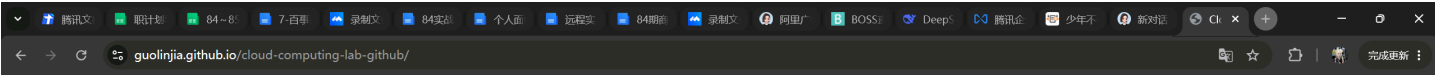
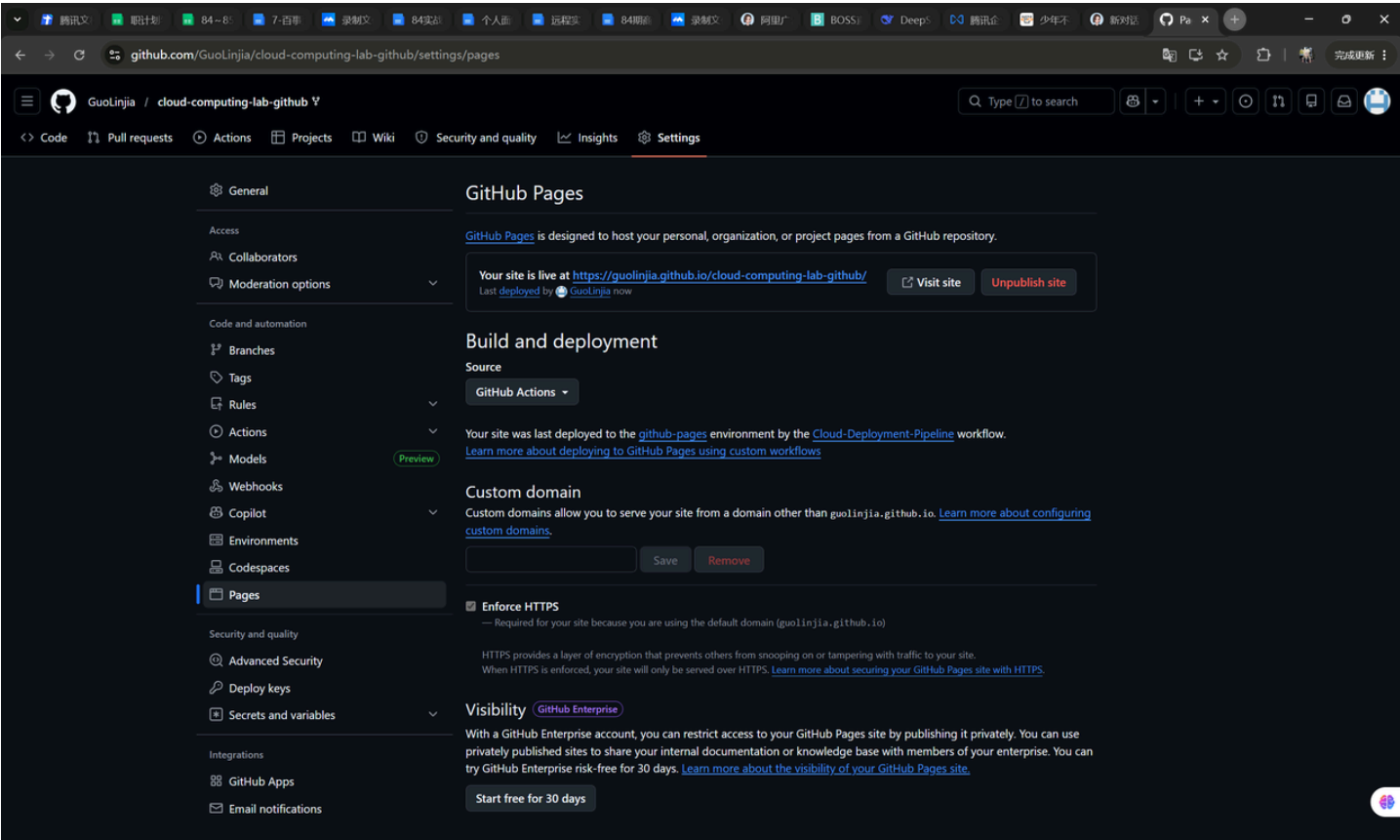




Step 5: Verification and Global Access

After the workflow was completed, I returned to **Settings > Pages** to obtain the public access URL of the website. I opened the URL in the browser and confirmed that the personalized page could be accessed normally on the public network.

My operation	Cloud concept reflected
Accessed and verified the online website through the generated URL.	The website is deployed based on the PaaS model. GitHub is responsible for server, infrastructure and public access management, and users only focus on content construction.



Step 6: Content Personalization and Multimedia Integration

On the basis of modifying the name, I further personalized the page: modified the CSS style to change the page background color and font style, and added an `<img>` tag to introduce GitHub Actions logo pictures. Each submission automatically triggered a new deployment, which verified the continuous deployment (CD) mechanism.

Main personalized operations:

- Update the student name and experimental information displayed on the page.
- Adjust the page theme color and layout style through the `<style>` code block.
- Add image elements to verify the cloud's processing capability of external resources.
- Confirm that each modification can trigger automatic redeployment.

4. Reflection

1. Where is your website stored?

The source code of the website is stored in the GitHub repository I Forked, and the deployed static page is hosted and published through GitHub Pages. The underlying construction, hosting and global distribution are all managed by GitHub's cloud infrastructure.

2. Did you have to install any software?

No. I completed all operations only through a browser and GitHub's online interface. I did not install a local web server, development tools or deployment software, because GitHub Actions and GitHub Pages completed all cloud-based deployment work.

3. Problems Encountered and Proposed Solutions

- Problem 1: I was not familiar with the GitHub Pages deployment source configuration at the beginning.

Solution: I checked the Settings > Pages page and selected GitHub Actions as the deployment source according to the experimental requirements.

- Problem 2: The online URL did not appear immediately after submitting the code.

Solution: I waited for the Cloud-Deployment-Pipeline in the Actions tab to finish running, and then checked the Pages page to obtain the URL.

- Problem 3: The updated content was not displayed in real time after access.

Solution: I refreshed the page after the workflow showed a green check mark to ensure the latest content was loaded.

Overall, this experiment made me intuitively feel the efficiency of cloud-native automated deployment. GitHub Actions and GitHub Pages realize the seamless connection of code modification, automatic construction, deployment and global access, and deeply understand the core advantages of PaaS services.